Lookup 메커니즘 (Lookup Mechanism)

개요

Lookup 메커니즘은 ZKEVM의 회로 간 데이터 공유와 검증을 가능하게 하는 핵심 기술입니다. Halo2의 lookup argument를 기반으로, 한 회로가 생성한 데이터를 다른 회로가 효율적으로 참조하고 검증할 수 있습니다.

주요 역할:

- 회로 간 데이터 일관성 보장
- 복잡한 연산 결과의 재사용
- 회로 크기 최적화 (중복 계산 제거)
- 다중 컬럼 데이터의 효율적 검증

파일 위치:

- `zkevm-circuits/src/table.rs` - 테이블 정의 및 LookupTable 트레이트
- `zkevm-circuits/src/evm_circuit/util/constraint_builder.rs` - Lookup 제약 조건 구축
- `zkevm-circuits/src/state_circuit/constraint_builder.rs` - State Circuit lookup
- 각 회로의 `configure` 메서드 - Lookup 구성

Halo2 Lookup API

기본 개념

Halo2의 lookup argument는 다음을 증명합니다:

```
∀ input ∈ Input: ∃ row ∈ Table: input = row
```

즉, 입력 집합의 모든 원소가 테이블에 존재함을 증명합니다.

lookup_any 메서드

Halo2는 `lookup_any` 메서드를 통해 lookup을 정의합니다:

```
meta.lookup_any("lookup name", |meta| {
// 입력 표현식들
let  input_exprs = vec![
input1.expr(),
input2.expr(),
// ...
];

// 테이블 표현식들
let  table_exprs = table.table_exprs(meta);

// (입력, 테이블) 쌍의 벡터 반환
vec![(input_expr, table_expr)]
});
```

파라미터:

- `name` : Lookup의 이름 (디버깅용)

- `closure` : 입력과 테이블 표현식을 정의하는 클로저
- 입력: `&mut VirtualCells<F>` - 셀 쿼리 인터페이스
- 출력: `Vec<(Expression<F>, Expression<F>)>` - (입력, 테이블) 쌍

동작 원리:

1. 입력 표현식과 테이블 표현식을 각각 계산
2. 모든 입력 값이 테이블에 존재하는지 검증
3. Plonk의 permutation argument를 사용하여 효율적으로 증명

LookupTable 트레이트

모든 테이블은 `LookupTable<F>` 트레이트를 구현합니다:

```
pub trait LookupTable<F: Field> {
/// 테이블의 모든 컬럼 반환
fn columns(&self) -> Vec<Column<Any>>;

/// Advice 컬럼만 반환
fn advice_columns(&self) -> Vec<Column<Advice>>;

/// Fixed 컬럼만 반환
fn fixed_columns(&self) -> Vec<Column<Fixed>>;

/// 컬럼 주석 반환
fn annotations(&self) -> Vec<String>;
```

```rust
/// Lookup에 사용되는 Expression 반환
fn table_exprs(&self, meta: &mut VirtualCells<F>) -> Vec<Expression<F>> {
self.columns()
.iter()
.map(|&column| meta.query_any(column, Rotation::cur()))
.collect()
}
}
```

주요 메서드:

- `columns()` : 테이블의 모든 컬럼 (Fixed + Advice)
- `table_exprs()` : Lookup에 사용할 표현식 생성
- `annotations()` : 각 컬럼의 이름 (디버깅용)

RLC를 통한 다중 컬럼 Lookup

문제: 다중 컬럼 검증

많은 테이블은 여러 컬럼으로 구성됩니다. 예를 들어 RW 테이블은 12개의 컬럼을 가집니다:

- `rw_counter` , `is_write` , `tag` , `id` , `address` , `field_tag` , `storage_key` , `value` , `value_prev` , `aux1` , `aux2`

Halo2의 `lookup_any` 는 단일 표현식 쌍만 지원하므로, 여러 컬럼을 하나의 값으로 압축해야 합니다.

해결책: Random Linear Combination (RLC)

RLC는 여러 값을 하나의 필드 원소로 압축하는 기법입니다:

$$RLC(v_0, v_1, \ldots, v_n) = v_0 + v_1 \cdot r + v_2 \cdot r^2 + \ldots + v_n \cdot r^n$$

여기서 r 은 challenge (무작위 값)입니다.

특징:

- **압축**: n개의 값을 1개의 필드 원소로 변환
- **고유성**: 높은 확률로 서로 다른 입력은 다른 RLC 값을 가짐
- **효율성**: 단일 lookup으로 다중 컬럼 검증 가능

RLC 구현

정의 위치: `zkevm-circuits/src/evm_circuit/util/common_gadget/rlc.rs`

```
/// RLC 계산 (little endian)
pub fn expr<F: Field, E: Expr<F>>(expressions: &[E], randomness: E) -> Express
let mut rlc = 0.expr();
let mut multiplier = 1.expr();
for expr in expressions.iter() {
rlc = rlc + expr.expr() * multiplier.clone();
multiplier = multiplier * randomness.expr();
}
rlc
}

/// RLC 계산 (big endian, 역순)
pub fn expr_rev<F: Field, E: Expr<F>>(expressions: &[E], randomness: E) -> Exp
```

```
let mut rlc = 0.expr();
for expr in expressions.iter().rev() {
rlc = rlc * randomness.expr() + expr.expr();
}
rlc
}
```

RLC 사용 예시

예시 1: Keccak Table Lookup

Keccak 테이블은 4개의 컬럼을 가집니다:

- `is_final` : 마지막 블록 여부
- `input_rlc` : 입력 데이터의 RLC
- `input_len` : 입력 길이
- `output_rlc` : 출력 해시의 RLC

Lookup 구성:

```
// EVM Circuit에서 Keccak 테이블 lookup
meta.lookup_any("keccak lookup", |meta| {
let input_rlc = rlc::expr(&input_bytes, challenges.keccak_input());
let output_rlc = rlc::expr(&output_bytes, challenges.keccak_input());

// 4개 컬럼을 하나의 RLC로 압축
```

```
let  input_expr = rlc::expr(
&[
1.expr(), // is_final = true
input_rlc,
input_len.expr(),
output_rlc,
],
challenges.lookup_input(),
);


let  table_expr = rlc::expr(
&keccak_table.table_exprs(meta),
challenges.lookup_input(),
);


vec![(input_expr, table_expr)]
});
```

단계별 설명:

1. 입력 바이트들을 `keccak_input` challenge로 RLC 계산
2. 출력 바이트들을 `keccak_input` challenge로 RLC 계산
3. 4개 컬럼 값을 `lookup_input` challenge로 다시 RLC 계산
4. 테이블의 4개 컬럼도 동일한 challenge로 RLC 계산

5. 두 RLC 값이 같은지 lookup으로 검증

예시 2: RW Table Lookup

RW 테이블은 12개의 컬럼을 가지므로 더 복잡한 RLC가 필요합니다:

```
// EVM Circuit에서 스택 pop 연산
pub(crate) fn stack_pop(&mut self, value: Expression<F>) {
self.stack_lookup(false.expr(), self.stack_pointer_offset.clone(), value);
}

fn stack_lookup(
&mut self,
is_write: Expression<F>,
stack_pointer_offset: Expression<F>,
value: Expression<F>,
) {
// RW 테이블의 12개 컬럼 구성
let rw_counter = self.curr.state.rw_counter.expr() + self.rw_counter_offset.clo
let is_write_expr = is_write;
let tag = RwTableTag::Stack.expr();
let id = self.curr.state.call_id.expr();
let address = self.curr.state.stack_pointer.expr() + stack_pointer_offset;
let field_tag = 0.expr();
let storage_key = 0.expr();
```

```
let value_expr = value;
let value_prev = 0.expr();
let aux1 = 0.expr();
let aux2 = 0.expr();


// 12개 컬럼을 RLC로 압축
let input_expr = rlc::expr(
&[
rw_counter,
is_write_expr,
tag,
id,
address,
field_tag,
storage_key,
value_expr,
value_prev,
aux1,
aux2,
],
self.challenges.lookup_input(),
);
```

```
let table_expr = rlc::expr(
&self.rw_table.table_exprs(meta),
self.challenges.lookup_input(),
);


self.add_lookup("stack lookup", vec![(input_expr, table_expr)]);
}
```

Challenge 사용

Challenge란?

Challenge는 증명 생성 과정에서 verifier가 prover에게 제공하는 무작위 값입니다. Halo2에서는 여러 단계(phase)에 걸쳐 challenge를 생성하고 사용할 수 있습니다.

특징:

- **무작위성**: Fiat-Shamir 변환을 통해 생성
- **보안성**: 예측 불가능하여 부정 증명 방지
- **단계별 생성**: Phase 1, Phase 2, Phase 3에서 각각 생성

Challenges 구조체

정의 위치: `zkevm-circuits/src/util/challenge.rs`

```
pub struct Challenges<T = Challenge> {
evm_word: T, // EVM word RLC용
keccak_input: T, // Keccak 입력 RLC용
```

```
lookup_input: T, // Lookup RLC용
}

impl<T: Clone> Challenges<T> {
pub fn evm_word(&self) -> T {
self.evm_word.clone()
}

pub fn keccak_input(&self) -> T {
self.keccak_input.clone()
}

pub fn lookup_input(&self) -> T {
self.lookup_input.clone()
}
}
```

Challenge 생성 시점

Challenges는 회로의 여러 phase에서 생성됩니다:

```
impl<F: Field> Challenges<Expression<F>> {
pub fn exprs(&self, meta: &mut ConstraintSystem<F>) -> Self {
Self {
```

```
evm_word: meta.challenge_usable_after(FirstPhase),
keccak_input: meta.challenge_usable_after(FirstPhase),
lookup_input: meta.challenge_usable_after(SecondPhase),
}
}
}
```

Phase별 사용:

- **FirstPhase**: 기본 제약 조건, 테이블 데이터 할당
- **SecondPhase**: RLC 계산, lookup 입력 준비
- `evm_word` : EVM word (256비트) RLC
- `keccak_input` : Keccak 입력/출력 RLC
- **ThirdPhase**: Lookup 수행
- `lookup_input` : 다중 컬럼 lookup RLC

Challenge 사용 패턴

패턴 1: 바이트 배열 RLC

```
// 32바이트 배열을 하나의 필드 원소로 압축
let bytes: [u8; 32] = [...];
let rlc_value = rlc::value(
bytes.iter().rev(), // Big endian
challenges.evm_word(),
```

```
);
```

패턴 2: 다중 컬럼 Lookup

```
// 여러 컬럼을 하나의 lookup으로 검증
let input_columns = vec![col1, col2, col3, col4];
let input_rlc = rlc::expr(&input_columns, challenges.lookup_input());

let table_columns = table.table_exprs(meta);
let table_rlc = rlc::expr(&table_columns, challenges.lookup_input());

meta.lookup_any("multi-column lookup", |meta| {
vec![(input_rlc, table_rlc)]
});
```

패턴 3: 조건부 Lookup

```
// 특정 조건에서만 lookup 수행
let condition = is_enabled.expr();
let input_rlc = condition.clone() * rlc::expr(&input_columns, challenges.lookup
let table_rlc = rlc::expr(&table_columns, challenges.lookup_input());

meta.lookup_any("conditional lookup", |meta| {
vec![(input_rlc, table_rlc)]
```

```
});
```

동작 원리:

- `condition = 0` 이면 `input_rlc = 0`
- 테이블의 첫 행을 모두 0으로 패딩
- 0은 항상 테이블에 존재하므로 lookup 성공

Lookup 구현 패턴

패턴 1: Fixed Table Lookup

Fixed 테이블은 컴파일 타임에 값이 결정되는 테이블입니다.

예시: 범위 검증 (u8 테이블)

```
// 테이블 정의
pub struct U8Table {
pub value: Column<Fixed>,
}

impl U8Table {
pub fn construct<F: Field>(meta: &mut ConstraintSystem<F>) -> Self {
Self {
value: meta.fixed_column(),
}
}
```

```rust
pub fn load(&self, layouter: &mut impl Layouter<F>) -> Result<(), Error> {
layouter.assign_region(
|| "u8 table",
|mut region| {
for i in 0..256 {
region.assign_fixed(
|| format!("u8[{}]", i),
self.value,
i,
|| Value::known(F::from(i as u64)),
)?;
}
Ok(())
},
)
}
}

// Lookup 사용
meta.lookup_any("byte range check", |meta| {
let byte_value = meta.query_advice(byte_column, Rotation::cur());
let table_value = meta.query_fixed(u8_table.value, Rotation::cur());
```

```
vec![(byte_value, table_value)]
});
```

용도:

- 범위 검증 (u8, u10, u16 등)
- 비트 연산 테이블 (AND, OR, XOR)
- 2의 거듭제곱 테이블

패턴 2: Dynamic Table Lookup

Dynamic 테이블은 실행 시 witness 데이터로 채워지는 테이블입니다.

예시: Bytecode Table Lookup

```
// 테이블 정의
pub struct BytecodeTable {
pub q_enable: Column<Fixed>,
pub code_hash: Column<Advice>,
pub tag: Column<Advice>,
pub index: Column<Advice>,
pub is_code: Column<Advice>,
pub value: Column<Advice>,
}

impl<F: Field> LookupTable<F> for BytecodeTable {
fn columns(&self) -> Vec<Column<Any>> {
```

```
vec![
self.q_enable.into(),
self.code_hash.into(),
self.tag.into(),
self.index.into(),
self.is_code.into(),
self.value.into(),
]
}

fn  annotations(&self) -> Vec<String> {
vec![
"q_enable".to_string(),
"code_hash".to_string(),
"tag".to_string(),
"index".to_string(),
"is_code".to_string(),
"value".to_string(),
]
}
}

// Lookup 사용 (EVM Circuit에서)
```

```rust
pub(crate) fn bytecode_lookup(
&mut self,
code_hash: Expression<F>,
index: Expression<F>,
is_code: Expression<F>,
value: Expression<F>,
) {
let input_expr = rlc::expr(
&[
1.expr(), // q_enable
code_hash,
BytecodeFieldTag::Byte.expr(), // tag
index,
is_code,
value,
],
self.challenges.lookup_input(),
);


let table_expr = rlc::expr(
&self.bytecode_table.table_exprs(meta),
self.challenges.lookup_input(),
);
```

```
self.add_lookup("bytecode lookup", vec![(input_expr, table_expr)]);
}
```

패턴 3: 조건부 Lookup with q_enable

많은 테이블은 `q_enable` 컬럼을 사용하여 유효한 행만 lookup에 포함시킵니다.

구현:

```
// 테이블 정의
pub struct MyTable {
pub q_enable: Column<Fixed>, // 1이면 유효한 행
pub col1: Column<Advice>,
pub col2: Column<Advice>,
}

// 테이블 할당
fn load_table(&self, layouter: &mut impl Layouter<F>, data: &[Data]) -> Result
layouter.assign_region(
|| "my table",
|mut region| {
// 첫 행은 0으로 패딩 (조건부 lookup용)
region.assign_fixed(|| "q_enable", self.q_enable, 0, || Value::known(F::zero()))
region.assign_advice(|| "col1", self.col1, 0, || Value::known(F::zero()))?;
```

```
region.assign_advice(|| "col2", self.col2, 0, || Value::known(F::zero()))?;


// 실제 데이터 할당
for (offset, item) in  data.iter().enumerate() {
let  offset = offset + 1;
region.assign_fixed(|| "q_enable", self.q_enable, offset, || Value::known(F::one
region.assign_advice(|| "col1", self.col1, offset, || Value::known(item.col1))?;
region.assign_advice(|| "col2", self.col2, offset, || Value::known(item.col2))?;
}
Ok(())
},
)
}


// Lookup 사용
meta.lookup_any("conditional lookup", |meta| {
let  condition = is_valid.expr();


let  input_expr = rlc::expr(
&[
condition.clone(), // q_enable과 매칭
input_col1,
input_col2,
```

```
],
challenges.lookup_input(),
);

let  table_expr = rlc::expr(
&[
meta.query_fixed(table.q_enable, Rotation::cur()),
meta.query_advice(table.col1, Rotation::cur()),
meta.query_advice(table.col2, Rotation::cur()),
],
challenges.lookup_input(),
);

vec![(input_expr, table_expr)]
});
```

동작 원리:

- `condition = 0` 이면 입력 RLC = 0
- 테이블의 첫 행 (q_enable=0)과 매칭
- `condition = 1` 이면 실제 데이터와 매칭

실제 회로에서의 Lookup 사용

EVM Circuit의 Lookup

EVM Circuit은 가장 많은 lookup을 수행하는 회로입니다:

1. RW Table Lookup

```
// 스택 연산
cb.stack_pop(value.expr());
cb.stack_push(result.expr());

// 메모리 연산
cb.memory_lookup(
1.expr(), // is_write
address.expr(), // address
value.expr(), // value
None, // value_prev
);

// 스토리지 연산
cb.account_storage_read(
address.expr(),
key.expr(),
value.expr(),
tx_id.expr(),
committed_value.expr(),
```

```
);
```

2. Bytecode Table Lookup

```
// 현재 opcode 조회
cb.bytecode_lookup(
code_hash.expr(),
program_counter.expr(),
is_code.expr(),
opcode.expr(),
);
```

3. Copy Table Lookup

```
// 데이터 복사 검증
cb.copy_table_lookup(
src_id.expr(),
CopyDataType::Memory.expr(),
dst_id.expr(),
CopyDataType::Memory.expr(),
src_addr.expr(),
src_addr_end.expr(),
dst_addr.expr(),
length.expr(),
```

```
rlc_acc.expr(),
rwc_inc.expr(),
);
```

4. Keccak Table Lookup

```
// Keccak 해시 검증
cb.keccak_table_lookup(
input_rlc.expr(),
input_len.expr(),
output_rlc.expr(),
);
```

State Circuit의 Lookup

State Circuit은 MPT 테이블을 lookup하여 상태 변경을 검증합니다:

```
// Account 마지막 접근 시 MPT lookup
self.condition(q.last_access(), |cb| {
cb.add_lookup("mpt_update lookup", vec![
(1.expr(), q.mpt_table.q_enable),
(q.rw_table.address, q.mpt_table.address),
(q.rw_table.storage_key, q.mpt_table.storage_key),
(q.mpt_proof_type(), q.mpt_table.proof_type),
(q.state_root(), q.mpt_table.new_root),
```

```
(q.state_root_prev(), q.mpt_table.old_root),
(q.value(), q.mpt_table.new_value),
(q.initial_value(), q.mpt_table.old_value),
]);
});
```

검증 내용:

- 주소와 스토리지 키
- 증명 타입 (존재/비존재)
- 상태 루트 변경 (old → new)
- 값 변경 (old → new)

Tx Circuit의 Lookup

Tx Circuit은 여러 테이블을 lookup합니다:

1. Sig Table Lookup (서명 검증)

```
// ECDSA 서명 검증
self.sig_table.add_lookup(
meta,
"sig table lookup",
|meta| {
vec![
(msg_hash_rlc.expr(), meta.query_advice(sig_table.msg_hash_rlc, Rotation::cur())
```

```
(sig_v.expr(), meta.query_advice(sig_table.sig_v, Rotation::cur())),
(sig_r_rlc.expr(), meta.query_advice(sig_table.sig_r_rlc, Rotation::cur())),
(sig_s_rlc.expr(), meta.query_advice(sig_table.sig_s_rlc, Rotation::cur())),
(recovered_addr.expr(), meta.query_advice(sig_table.recovered_addr, Rotation::cu
(1.expr(), meta.query_advice(sig_table.is_valid, Rotation::cur())),
]
},
);
```

2. RLP Table Lookup (RLP 인코딩 검증)

```
// 트랜잭션 RLP 인코딩 검증
self.rlp_table.add_lookup(
meta,
"rlp table lookup",
|meta| {
vec![
(tx_id.expr(), meta.query_advice(rlp_table.tx_id, Rotation::cur())),
(format.expr(), meta.query_advice(rlp_table.format, Rotation::cur())),
(rlp_tag.expr(), meta.query_advice(rlp_table.rlp_tag, Rotation::cur())),
// ...
]
},
```

```
);
```

3. Keccak Table Lookup (트랜잭션 해시)

```
// 트랜잭션 해시 계산 검증
self.keccak_table.add_lookup(
meta,
"tx hash lookup",
|meta| {
vec![
(1.expr(), meta.query_advice(keccak_table.is_final, Rotation::cur())),
(tx_rlp_rlc.expr(), meta.query_advice(keccak_table.input_rlc, Rotation::cur())),
(tx_rlp_len.expr(), meta.query_advice(keccak_table.input_len, Rotation::cur())),
(tx_hash_rlc.expr(), meta.query_advice(keccak_table.output_rlc, Rotation::cur))
]
},
);
```

Lookup 최적화 기법

1. 테이블 크기 최소화

문제: 큰 테이블은 증명 생성 시간과 크기를 증가시킵니다.

해결책:

- 필요한 데이터만 테이블에 포함
- 중복 제거
- 효율적인 인코딩 사용

예시: Bytecode Table

```
// 각 바이트코드를 언롤링하여 저장
// 중복된 코드 해시는 한 번만 저장
pub fn assign_bytecode_table(
&self,
layouter: &mut impl Layouter<F>,
bytecodes: &[Bytecode],
challenges: &Challenges<Value<F>>,
) -> Result<(), Error> {
// 코드 해시로 중복 제거
let unique_bytecodes: BTreeMap<H256, &Bytecode> = bytecodes
.iter()
.map(|b| (b.hash(), b))
.collect();

// 언롤링된 바이트만 저장
for bytecode in unique_bytecodes.values() {
for (index, byte) in bytecode.code.iter().enumerate() {
// 각 바이트를 한 행에 할당
```

```
}
}
}
```

2. RLC 재사용

문제: 같은 데이터에 대해 여러 번 RLC를 계산하면 비효율적입니다.

해결책:

- RLC 값을 미리 계산하여 저장
- 여러 lookup에서 재사용

예시: Word RLC

```
// 256비트 word를 32바이트로 분해하여 RLC 계산
pub struct Word<F> {
pub cells: [Cell<F>; 32], // 32 bytes
pub rlc: Cell<F>, // 미리 계산된 RLC
}

impl<F: Field> Word<F> {
pub fn assign(
&self,
region: &mut CachedRegion<'_, '_, F>,
offset: usize,
value: U256,
```

```
challenges: &Challenges<Value<F>>,
) -> Result<(), Error> {
let bytes = value.to_le_bytes();

// 각 바이트 할당
for (cell, &byte) in self.cells.iter().zip(bytes.iter()) {
cell.assign(region, offset, Value::known(F::from(byte as u64)))?;
}

// RLC 계산 및 할당
let rlc = challenges.evm_word().map(|challenge| {
rlc::value(bytes.iter().rev(), challenge)
});
self.rlc.assign(region, offset, rlc)?;

Ok(())
}
}
```

3. Lookup 횟수 감소

문제: 불필요한 lookup은 회로 크기를 증가시킵니다.

해결책:

- 조건부 lookup 사용

- 여러 lookup을 하나로 통합
- 캐싱 활용

예시: 조건부 Bytecode Lookup

```
// PUSH 명령어일 때만 다음 바이트들을 lookup
cb.condition(is_push.expr(), |cb| {
for i in 0..push_size {
cb.bytecode_lookup(
code_hash.expr(),
(program_counter.expr() + i.expr() + 1.expr()),
0.expr(), // is_code = false (push data)
push_data[i].expr(),
);
}
});
```

4. Phase 최적화

문제: 모든 컬럼을 SecondPhase에 두면 증명 생성이 느려집니다.

해결책:

- FirstPhase: 기본 데이터, Fixed 컬럼
- SecondPhase: RLC 계산이 필요한 컬럼만
- ThirdPhase: Lookup 전용 컬럼

예시: 컬럼 Phase 설정

```rust
pub struct MyTable {
pub q_enable: Column<Fixed>, // FirstPhase (Fixed)
pub id: Column<Advice>, // FirstPhase
pub value: Column<Advice>, // FirstPhase
pub rlc: Column<Advice>, // SecondPhase (RLC 계산)
}

impl MyTable {
pub fn construct<F: Field>(meta: &mut ConstraintSystem<F>) -> Self {
Self {
q_enable: meta.fixed_column(),
id: meta.advice_column(),
value: meta.advice_column(),
rlc: meta.advice_column_in(SecondPhase), // SecondPhase 명시
}
}
}
```

동작 예시

예시 1: 단순 범위 검증

시나리오: 바이트 값이 0~255 범위에 있는지 검증

테이블:

| value |

|-------|

| 0 |

| 1 |

| 2 |

| ... |

| 255 |

Lookup:

```
meta.lookup_any("byte range check", |meta| {
let  byte = meta.query_advice(byte_column, Rotation::cur());
let  table = meta.query_fixed(u8_table.value, Rotation::cur());
vec![(byte, table)]
});
```

검증:

- 입력: `byte = 42`
- 테이블에서 42 찾기 → 존재 ✓
- 입력: `byte = 256`
- 테이블에서 256 찾기 → 존재하지 않음 ✗ (증명 실패)

예시 2: Keccak 해시 검증

시나리오: `keccak256("hello")` 계산 검증

입력 데이터:

- `input` : "hello" = [0x68, 0x65, 0x6c, 0x6c, 0x6f]
- `input_len` : 5
- `output` : 0x1c8aff950685c2ed4bc3174f3472287b56d9517b9c948127319a09a7a36deac8

Keccak Table 행:

is_final	input_rlc	input_len	output_rlc
1	RLC([0x68,0x65,0x6c,0x6c,0x6f])	5	RLC(0x1c8a...eac8)

Lookup 과정:

1. **입력 RLC 계산:**

```
let  r = challenges.keccak_input();

input_rlc = 0x68 + 0x65*r + 0x6c*r² + 0x6c*r³ + 0x6f*r⁴
```

2. **출력 RLC 계산:**

```
output_rlc = RLC(0x1c8a...eac8의 32바이트)
```

3. **4개 컬럼을 하나의 RLC로 압축:**

```
let  r2 = challenges.lookup_input();

input_expr = 1 + input_rlc*r2 + 5*r2² + output_rlc*r2³

table_expr = 1 + table.input_rlc*r2 + table.input_len*r2² + table.output_rlc*r2³
```

4. **Lookup 검증:**

- `input_expr == table_expr` ✓
- Keccak 계산이 올바름을 증명

예시 3: 스택 연산 검증

시나리오: Call ID 5에서 스택에 값 push 후 pop

RW Table:

| rw_counter | is_write | tag | id | address | value | ... |

|----------|----------|----|---|---|---|---|

| 100 | 1 | Stack | 5 | 1023 | 0x1234 | ... |

| 101 | 0 | Stack | 5 | 1023 | 0x1234 | ... |

EVM Circuit에서 Lookup:

1. **Push 연산 (rw_counter=100):**

```
// 입력 구성
let  input = [
100, // rw_counter
1, // is_write
Stack.tag(), // tag = 2
5, // call_id
1023, // stack_pointer
0, // field_tag
0, // storage_key
0x1234, // value
```

```
0, // value_prev
0, // aux1
0, // aux2
];

// RLC 계산
let  r = challenges.lookup_input();
input_rlc = 100 + 1*r + 2*r² + 5*r³ + 1023*r⁴ + ... + 0x1234*r⁷ + ...

// Lookup
lookup(input_rlc, rw_table_rlc) // ✓ 테이블에 존재
```

2. **Pop 연산 (rw_counter=101):**

```
// 입력 구성
let  input = [
101, // rw_counter
0, // is_write (read)
Stack.tag(),
5,
1023,
0,
0,
```

```
0x1234, // 읽은 값
0x1234, // value_prev (이전 값)
0,
0,
];


// Lookup
lookup(input_rlc, rw_table_rlc) // ✓ 테이블에 존재
```

검증 효과:

- Push한 값과 Pop한 값이 일치함을 보장
- 스택 포인터가 올바르게 관리됨을 보장
- RW 카운터가 순차적으로 증가함을 보장

예시 4: 조건부 Lookup

시나리오: PUSH 명령어일 때만 다음 바이트들을 lookup

Bytecode:

```
PUSH2 0x1234 // PC=0: PUSH2 (0x61)
// PC=1: 0x12
// PC=2: 0x34
ADD // PC=3: ADD (0x01)
```

Bytecode Table:

| code_hash | tag | index | is_code | value |

|----------|-----|-------|---------|-------|

| 0xabcd... | Byte | 0 | 1 | 0x61 |

| 0xabcd... | Byte | 1 | 0 | 0x12 |

| 0xabcd... | Byte | 2 | 0 | 0x34 |

| 0xabcd... | Byte | 3 | 1 | 0x01 |

EVM Circuit Lookup:

1. **PC=0 (PUSH2 opcode):**

```
// Opcode lookup

cb.bytecode_lookup(

code_hash,

0, // index

1, // is_code

0x61, // PUSH2

); // ✓


// Push data lookup (조건부)

let  is_push = (opcode == 0x61); // true

cb.condition(is_push, |cb| {

cb.bytecode_lookup(code_hash, 1, 0, 0x12); // ✓

cb.bytecode_lookup(code_hash, 2, 0, 0x34); // ✓

});
```

2. **PC=3 (ADD opcode):**

```
// Opcode lookup
cb.bytecode_lookup(
code_hash,
3, // index
1, // is_code
0x01, // ADD
); // ✓


// Push data lookup (조건부)
let  is_push = (opcode == 0x01); // false
cb.condition(is_push, |cb| {
// 실행되지 않음 (is_push = 0)
// input_rlc = 0 * rlc(...) = 0
// 테이블의 첫 행 (모두 0)과 매칭
});
```

검증 효과:

- PUSH 명령어의 데이터가 올바른지 검증
- 일반 opcode는 불필요한 lookup 수행하지 않음
- 조건부 lookup으로 회로 효율성 향상

성능 고려사항

Lookup 비용

Lookup은 회로 크기와 증명 시간에 영향을 미칩니다:

비용 요소:

1. **테이블 크기**: 테이블 행 수에 비례
2. **Lookup 횟수**: 각 lookup은 제약 조건 추가
3. **컬럼 수**: RLC 계산 복잡도 증가
4. **Phase**: SecondPhase 컬럼은 FirstPhase보다 느림

최적화 전략:

- 작은 테이블 사용
- 조건부 lookup으로 불필요한 lookup 제거
- RLC 재사용
- 적절한 Phase 선택

메모리 사용

테이블 크기 예시:

| 테이블 | 행 수 | 컬럼 수 | 메모리 |

| --- | --- | --- | --- |

| U8 Table | 256 | 1 | 작음 |

| U16 Table | 65,536 | 1 | 중간 |

| Bytecode Table | ~100K | 6 | 큼 |

| RW Table | ~1M | 12 | 매우 큼 |

| Keccak Table | ~10K | 4 | 중간 |

메모리 최적화:

- 필요한 데이터만 테이블에 포함

- 중복 제거
- 압축 인코딩 사용

병렬화

Lookup은 병렬로 검증할 수 있습니다:

```
// 여러 lookup을 동시에 수행
meta.lookup_any("lookup1", |meta| { /* ... */ });
meta.lookup_any("lookup2", |meta| { /* ... */ });
meta.lookup_any("lookup3", |meta| { /* ... */ });

// 각 lookup은 독립적으로 검증됨
```

병렬화 이점:

- 증명 생성 시간 단축
- 멀티코어 활용
- 메모리 효율성